List of Contributors

- **Emre İnci:** Brief Project Description, System Architecture & Technology Stack, 3. Use Case Support: Use Case 1 (Authentication)
- **Orhan Faruk Demir:** 2. Implementation Details: Codebase Structure, 2. Implementation Details: Component & Visual Interfaces, 3. Use Case Support: Use Case 2 (Mood Entry)
- **Bekir Ata Yıldırım:** 2. Implementation Details: Key Implementations, 3. Use Case Support: Use Case 3 & 4 (Recommendation, History), 4. Design Decisions (Technology Comparisons)

Section / Task	Responsible Team Member
Design Doc: Brief Project Description	Emre İnci
Design Doc: System Architecture & Technology Stack	Emre İnci
Design Doc: 3. Use Case Support: Use Case 1 (Authentication)	Emre İnci
Design Doc: 2. Implementation Details: Codebase Structure	Orhan Faruk Demir
Design Doc: 2. Implementation Details: Component & Visual Interfaces	Orhan Faruk Demir
Design Doc: 3. Use Case Support: Use Case 2 (Mood Entry)	Orhan Faruk Demir
Design Doc: 2. Implementation Details: Key Implementations	Bekir Ata Yıldırım

Design Doc: 3. Use Case Support: Use Case 3 & 4 (Recommendation, History)	Bekir Ata Yıldırım
Design Doc: 4. Design Decisions (Technology Comparisons)	Bekir Ata Yıldırım
QA Plan: QA Strategy & Factors	Bekir Ata Yıldırım
QA Plan: Test Plan & Cases	Orhan Faruk Demir
Final Review & Formatting	Emre İnci

- ## Brief Project Description

DailyBeat, kullanıcıların duygusal farkındalıklarını artırmayı ve anlık ruh hallerine en uygun müzik eşleşmesini sağlamayı hedefleyen web tabanlı bir uygulamadır. Kullanıcılar günlük duygu durumlarını (mutlu, üzgün, enerjik vb.) sisteme girer; sistem bu verileri işleyerek Spotify veya YouTube API entegrasyonu üzerinden kişiselleştirilmiş şarkı önerileri sunar. Hedef kitle, stres yönetimi yapmak isteyen öğrenciler, beyaz yakalılar ve yeni müzik keşfetmek isteyen kullanıcılardır.

System Architecture

DailyBeat, modern web standartlarına uygun, bileşen tabanlı bir **Client-Server (İstemci-Sunucu)** mimarisi kullanmaktadır.

- **Frontend (Client):** Kullanıcı arayüzü ve etkileşimi yönetir.
- **Backend (Serverless/API):** İş mantığını (Business Logic) yürütür, dış API'ler ile haberleşir.
- **Database:** Kullanıcı verilerini ve duygu geçmişini tutar.

Technology Stack

Projede kullanılan temel teknolojiler şunlardır:

- **Frontend Framework:** Next.js (React tabanlı, Responsive UI için)
- **Backend Environment:** Node.js (Next.js API Routes)
- **Database:** Firebase (Statik ve Real time verilerin saklanması ve Auth işlemleri için)

- **External APIs:** Spotify Web API / YouTube Data API
- **Version Control:** Git & GitHub
- **IDE:** Visual Studio Code

2. Implementation Details

Codebase Structure (Kod Yapısı)

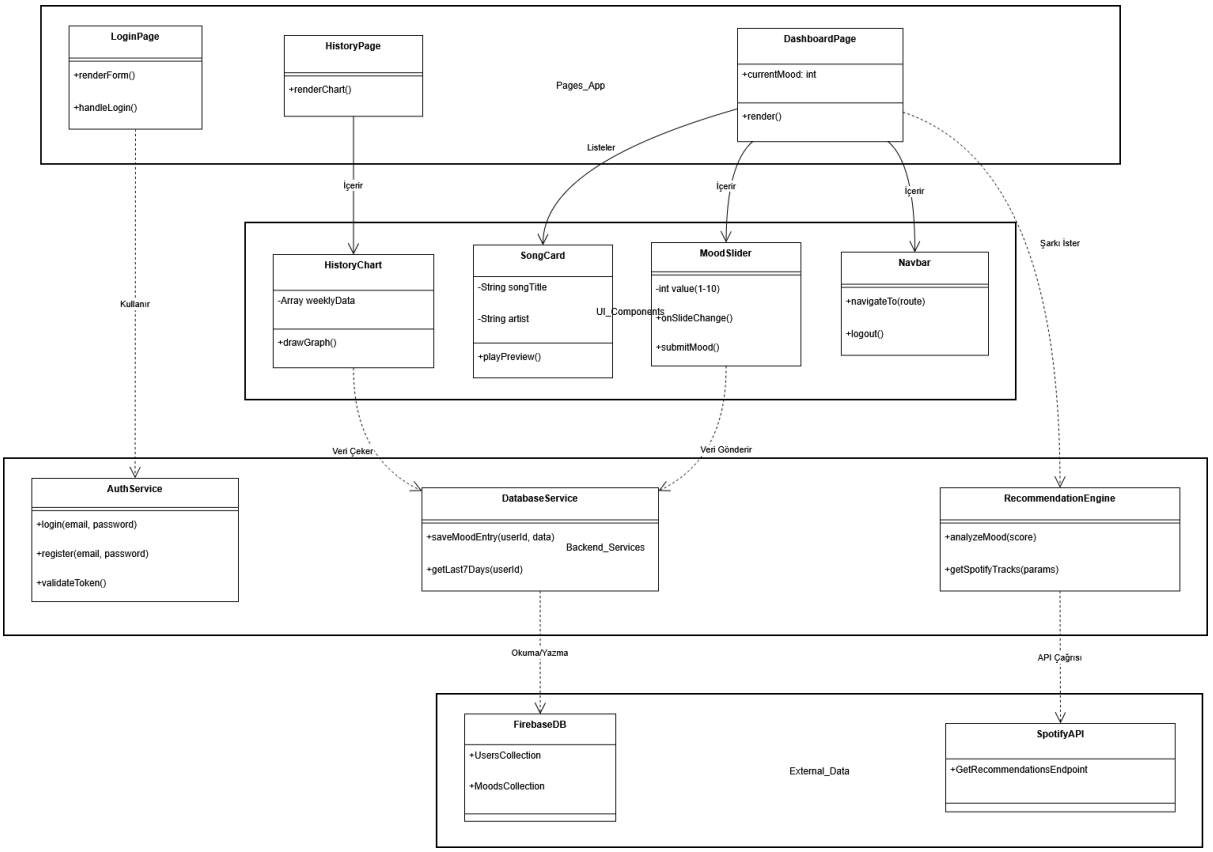
Projemiz, dosyaların düzenli durması için şu klasörlere ayrılmıştır:

- **/app (Sayfalar):** Kullanıcının gezindiği ekranlar buradadır. (Örneğin: Giriş Ekranı, Profil Sayfası, Anasayfa).
- **/components (Parçalar):** Sayfaların içinde bulunan özelliklerdir. Örneğin menü çubuğu, duygu durumu değerlendirme ekranı, geçmiş duygu durumu değişimi vb.
- **/lib (Arka Plan İşleri):** Ekranın arkasında çalışan, Spotify ve Veritabanı ile konuşan "beyin" kısmıdır.
- **/public (Görseller):** Projedeki logolar, resimler ve ikonların durduğu depodur.
- **/styles (Tasarım):** Sitenin renklerini, yazı tiplerini ve güzelleştirmelerini yaptığımız kısımdır.

Key Implementations

- **Mood-Based Recommendation Engine:** Kullanıcının girdiği 1-10 arası puanı veya emojiyi , API'den çekilen şarkıların kişilerin o an sahip olduğu duygu durumlarına uygun olarak önerilmesi ve bu önerinin algoritmasının kullanılması. Şarkı verilerinin etiketlenmesi.
- **Authentication Service:** Firebase Authentication kullanılarak e-posta ve şifre ile güvenli giriş/kayıt işlemlerini yöneten modüldür.
- **History Visualization:** Veritabanından çekilen tarih etiketli duygu verilerini grafiksel olarak kullanıcıya farklı zaman dilimlerinde neleri daha çok dinlediğinin gösterilmesi ve duygu değişimini takip etmesi sağlanabilir.

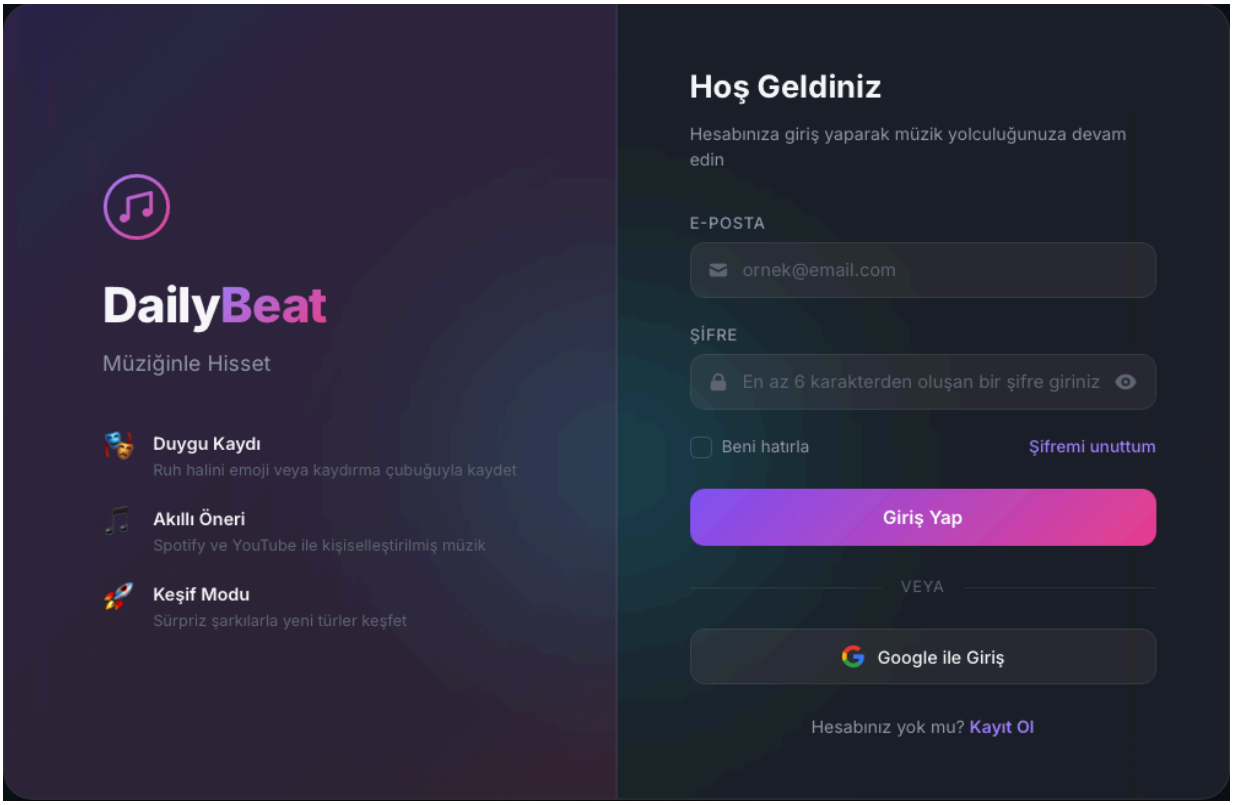
Component Interfaces



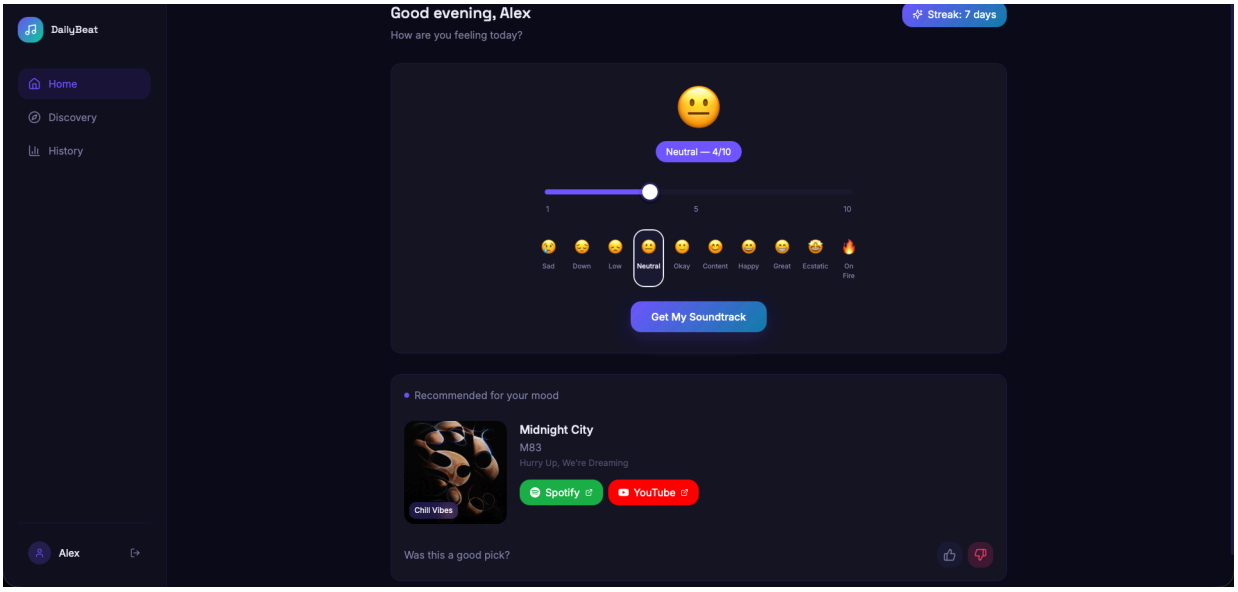
Visual Interfaces

Figma kullanılarak web sitesine ait interfaceler için örnek arayüzler hazırlanmış ve bunlar doğrultusunda web sitesi dizayn edilecektir.

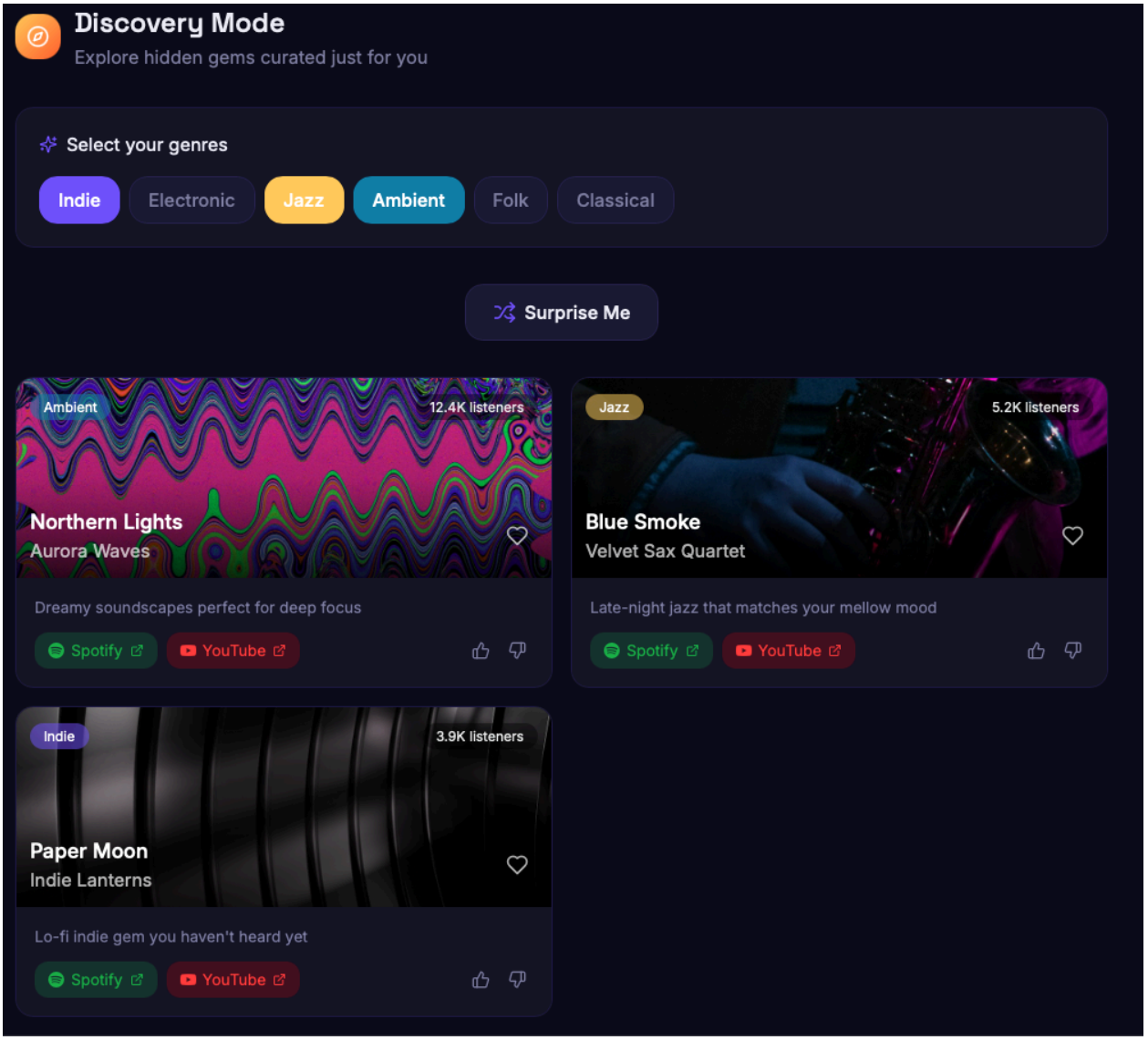
1. User Login Interface



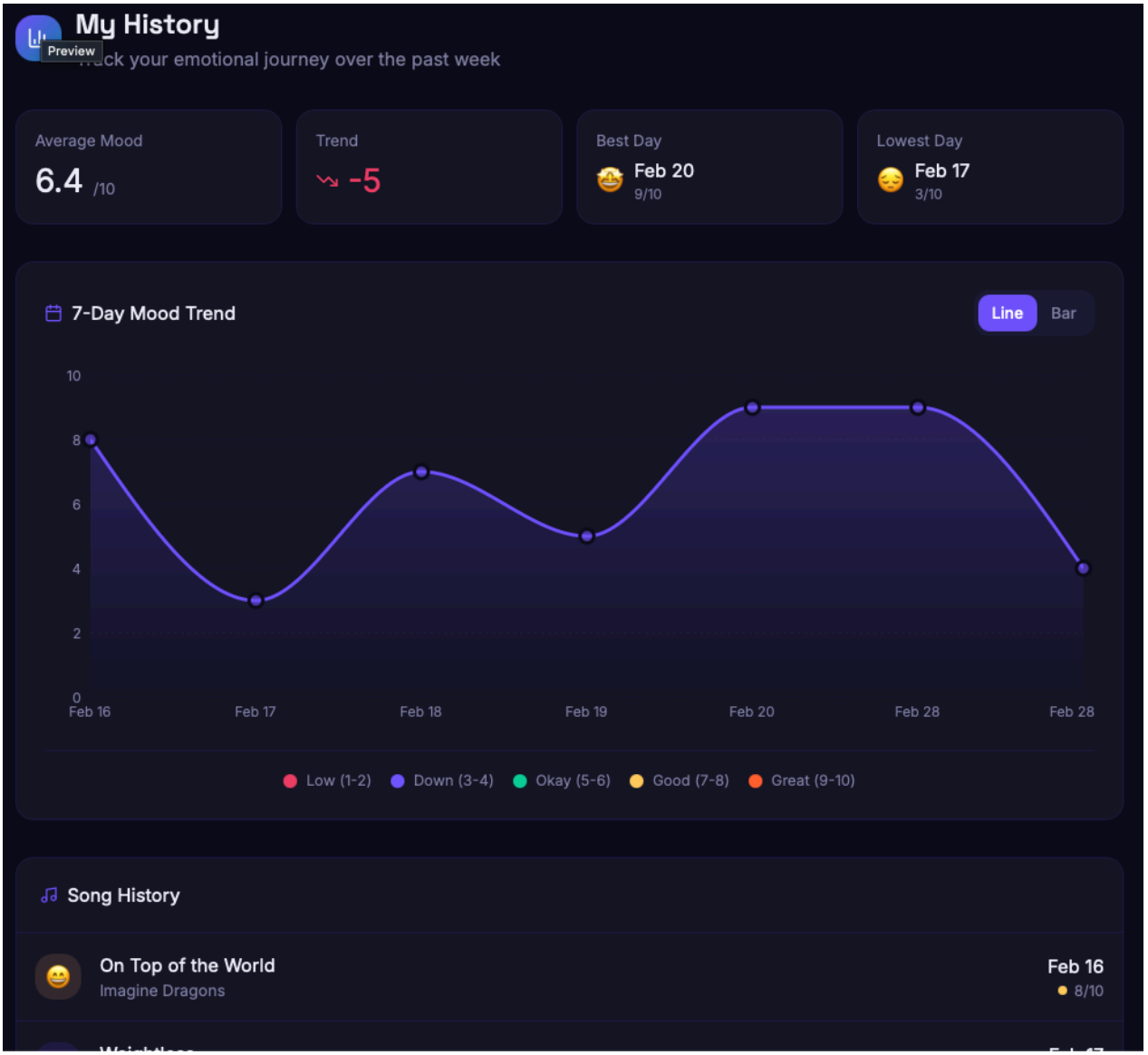
2. Home Page Interface



3. **Discovery Mode Page Interface**



4. **History Page Interface**



3. Use Case Support in Design

Projenin temel işlevlerini kapsayan ve Requirements Document içinden seçilen 4 ana Use Case (Kullanım Senaryosu) aşağıda listelenmiştir. Bu senaryolar final sunumunda canlı olarak demo edilecektir:

1. Kullanıcı Kaydı ve Girişi (Authentication)
2. Duygu Durumu Girişi (Mood Entry)
3. Müzik Önerisi Alma (Recommendation)
4. Geçmişi Görüntüleme (History Visualization)

3.1 Kullanıcı Kaydı ve Girişi (Authentication)

- **Requirement Mapping:** Sistem, kullanıcıların e-posta ve şifre ile güvenli bir şekilde kayıt olmasını ve giriş yapmasını sağlamalıdır.
- **Use Case Design:**
  - **System Architecture Support:** Frontend (Next.js) üzerinde oluşturulan giriş/kayıt formu, kimlik doğrulama işlemleri için doğrudan Firebase Authentication servisi ile haberleşir.
  - **Data Flow & Interactions:**

    1. Kullanıcı arayüzden e-posta ve şifre bilgilerini girip formu submit eder.

    2. Frontend, bu bilgileri Firebase Auth API'sine güvenli bir istek (request) olarak iletir.

    3. Firebase verileri doğrular. Eğer şifre yanlışsa frontend'e hata kodu döner ve ekranda uyarı çıkar. İşlem başarılıysa bir JWT (JSON Web Token) döner.

    4. Sistem bu token'ı tarayıcıda tutarak State'i "Logged In" (Giriş Yapıldı) olarak günceller ve kullanıcıyı Dashboard'a yönlendirir.

3.2 Duygu Durumu Girişi (Mood Entry)

- **Requirement Mapping:** Kullanıcı, bildirim üzerinden veya uygulama içinden o anki duygu durumunu (1-10 arası puan veya emoji ile) seçebilmelidir." VE "Sistem, kullanıcının seçtiği duygu durumunu veritabanına tarih ve saat etiketiyle kaydetmelidir.
- **Use Case Design:**
  - **System Architecture Support:** Kullanıcının etkileşime girdiği UI bileşenleri (Slider/Emoji grid), React State yönetimi ile anlık veriyi tutar ve Backend API aracılığıyla Firebase Firestore (Veritabanı) servisine aktarır.
  - **Data Flow & Interactions:**
    1. Kullanıcı Dashboard üzerinde "Mood Slider" bileşenini kaydırdığında (örn: 8/10), React'teki yerel State güncellenir.
    2. "Kaydet" butonuna basıldığında, Next.js API Route'una bir POST isteği atılır. Bu istek userId, moodScore ve timestamp (zaman damgası) verilerini içerir.
    3. Backend, bu veriyi yapılandırarak Firebase Firestore'daki "mood_logs" koleksiyonuna döküman olarak yazar.
    4. Veritabanından gelen "Başarılı" yanıtı üzerine UI güncellenir ve kullanıcıya onay mesajı gösterilir.

3.3 Müzik Önerisi Alma (Recommendation)

- **Requirement Mapping:** Sistem, girilen duygu durumuna uygun olarak harici bir API (Spotify/YouTube) üzerinden şarkı önermelidir.
- **Use Case Design:**
  - **System Architecture Support:** Sistemin "beyni" olan bu kısımda Node.js (Backend) devreye girer. Veritabanındaki duygu puanını, Spotify Web API'nin anlayacağı müzikal parametrelere (tempo, valence, energy) dönüştürür.

- **Data Flow & Interactions:**
    1. Duygu girişi başarılı olduktan sonra sistem, backend'deki /api/recommendation endpoint'ini tetikler.
    2. Backend algoritması duygu puanını işler (Örn: Mood=8 ise target_valence=0.8 ve target_energy=0.7 olarak maplenir).
    3. Backend, bu parametrelerle Spotify API'sine bir GET isteği atar.
    4. Spotify'dan dönen JSON formatındaki karmaşık veri listesi backend'de filtrelenir (Sadece şarkı adı, sanatçı, kapak resmi ve dinleme linki alınır).
    5. Temizlenen bu liste Frontend'e gönderilir ve kullanıcı arayüzünde "Önerilen Şarkılar" kartları olarak render edilir (çizilir).

3.4 Geçmişi Görüntüleme (History Visualization)

- **Requirement Mapping:** Kullanıcı, 'Geçmişim' sayfasında son 7 güne ait duygu durumlarını liste veya grafik halinde görüntüleyebilmelidir.
- **Use Case Design:**
    - **System Architecture Support:** Veritabanı (Firestore) okuma işlemleri ve Frontend tarafında verinin bir grafik kütüphanesi (örn: Chart.js veya Recharts) ile görselleştirilmesi mimarisine dayanır.
    - **Data Flow & Interactions:**
        1. Kullanıcı "Geçmişim" (My History) sayfasına tıkladığında, sayfa yüklenirken backend'e bir GET isteği atılır.
        2. Backend, Firebase Firestore'da userId eşleşmesi yapar ve sorguyu timestamp parametresine göre filtreleyerek sadece son 7 günün verilerini çeker.
        3. Dönen ham veriler (tarih ve 1-10 arası puanlar), frontend'deki grafik bileşeninin okuyabileceği X ve Y ekseni formatına (Array of objects) dönüştürülür.
        4. Grafik bileşeni state üzerinden bu yeni diziyi alır ve ekranda tarih eksenli bir eğri/çizgi grafiği olarak kullanıcıya sunar.

4. Design Decisions

Technology Comparisons

1. **Database: Firebase (NoSQL) vs. PostgreSQL (Relational SQL)**
    - *Karar:* **Firebase** seçilmiştir.
    - *Gerekçe:* Projenin hızlı prototiplenmesi gerekmektedir. Firebase'in sağladığı gerçek zamanlı veri senkronizasyonu ve hazır Authentication altyapısı, backend geliştirme eforunu optimize etmek için daha uygundur.
2. **Frontend: Next.js vs. Vanilla React**
    - *Karar:* **Next.js** seçilmiştir.
    - *Gerekçe:* API rotalarını (Backend logic) aynı proje içinde barındırabilmesi ve sayfa yönlendirmelerini (Routing) otomatik yönetmesi sebebiyle geliştirme sürecini hızlandırır.